Yelp AI Suite

pipeline analitica e sistema di raccomandazione ibrido su larga scala

Corso: Sistemi Intelligenti per Internet
Studente: Valerio Fiorentini (matricola 641622)
Codice sorgente: https://github.com/valeriofiorentini/yelp-bigdata-mining-recommender
Yelp Open *Dataset(JSON)*: https://business.yelp.com/data/resources/open-dataset/

Sintesi

La presente relazione si propone di descrivere lo sviluppo di Yelp AI Suite, una *pipeline* analitica integrata su larga scala costruita sullo Yelp Academic *Dataset* (oltre 150.000 attività commerciali e milioni di recensioni). Il sistema copre l'intero ciclo di lavoro: *ingestion*, analisi esplorativa e geospaziale, *Social Network Analysis, text mining* con classificazione del sentiment, e un motore di raccomandazione multi-paradigma, il tutto esposto in una *dashboard* web interattiva.

Si riportano come risultati principali: una compressione dei dati superiore al 95%, un *F1-score* di 0,97 sulla classificazione del *sentiment* (*AUC* 0,985) e un errore medio di 0,76 stelle (*MAE*) per il *recommender* neurale, con un compromesso misurabile tra latenza (*KNN* ~55 ms) e accuratezza (*Deep Learning* ~436 ms).

1. Introduzione e obiettivi

Lo scopo del progetto è di sviluppare una suite di intelligenza artificiale per l'*Information Retrieval*, per il *Text Processing* e per la *Social Search*, *app*licata allo Yelp Academic *Dataset*. Tale progetto si concentra su analisi di topic e creazione di un *Recommender System* ibrido.

Si riassumono gli obiettivi tecnici nei seguenti punti:

- Efficienza: elaborazione del *dataset* in modo ottimale, evitando la saturazione della memoria.
- **Analisi:** disponibilità di esplorazione geospaziale, analisi delle reti sociali e dei testi delle recensioni.
- **Raccomandazione:** presenza di suggerimenti di ristoranti e locali affini ai gusti dell'utente con tre approcci complementari.
- Fru*ibi*lità: access*ibi*lità resa possibile tramite una *dashboard web* interattiva e reattiva.

2. Architettura del sistema

La *pipeline* è modulare e si articola in cinque fasi: (1) *Ingestion*, (2) *EDA* e *SNA*, (3) *NLP*, (4) Recommender, (5) *Business Intelligence*. Gli artefatti intermedi (file *Parquet*, modelli serializzati) sono salvati in un *artifact layer* condiviso su *Google Drive*, che permette di eseguire ogni fase in modo indipendente.

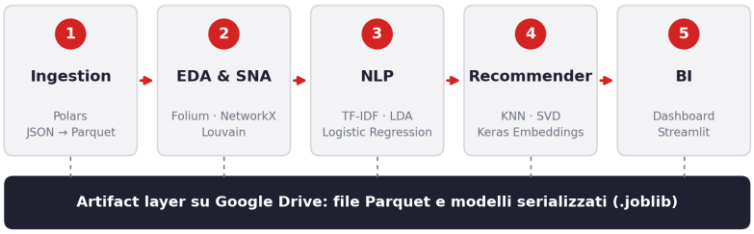


Figura 1 - Architettura modulare della *pipeline* in cinque fasi

2.1 Stack tecnologico

Le librerie Python vengono scelte per bilanciare il carico computazionale, evitare crash di memoria e garantire un'esplorazione in tempo reale.

Libreria	Ruolo	Impiego nel progetto
Polars	Ingestion e manipolazione dati	Scritta in Rust. L'architettura colonnare e la Lazy API riducono i tempi da minuti a frazioni di secondo.
Scikit-Learn	Machine Learning classico	Logistic Regression (sentiment), Cosine Similarity (content-based), Truncated SVD (collaborative filtering).
NetworkX	Social Network Analysis	Modellazione a grafo, Degree Centrality e rilevamento di community (algoritmo di Louvain).
TensorFlow / Keras	Deep Learning	Neural Collaborative Filtering con strati di Embedding densi.
Folium / Streamlit-Folium	Mapping spaziale	Heatmap e cluster geolocalizzati (es. area di Philadelphia).
Streamlit	Interfaccia web	Trasforma il lavoro di ML in una dashboard interattiva per l'utente finale.

Tabella 1 - Componenti dello stack tecnologico

3. Acquisizione e preparazione dei dati

3.1 Caricamento del *dataset*

I dati testuali e strutturati vengono estratti e preparati (*Text Acquisition*), mantenendo le colonne essenziali e i locali *app*artenenti alla categoria *Food/Restaurants*. Questo *app*roccio difatti serve a produrre una base dati solida per le successive fasi di *Information Retrieval* e *Text Processing*.

3.2 Pulizia e *preprocessing*

I dati testuali e categorici vengono normalizzati; descrizioni e categorie prive di classificazione sono rimosse per evitare rumore. Sul testo delle recensioni si eliminano le *stop-words* e si *app*lica la lemmatizzazione; i *rating* (stelle) risultano *bi*lanciati prima del partizionamento in *training* e *test set*.

3.3 Analisi esplorativa e geospaziale

L'*EDA* ha evidenziato dinamiche temporali chiare: i picchi di affluenza, individuati con matrici giorno/ora, si concentrano tra il venerdì e il sabato sera. L'analisi cartografica ha permesso di confrontare la struttura delle reti urbane (ad esempio, la griglia ortogonale di Las Vegas rispetto alle città europee) tramite m*appe* interattive *Folium*.

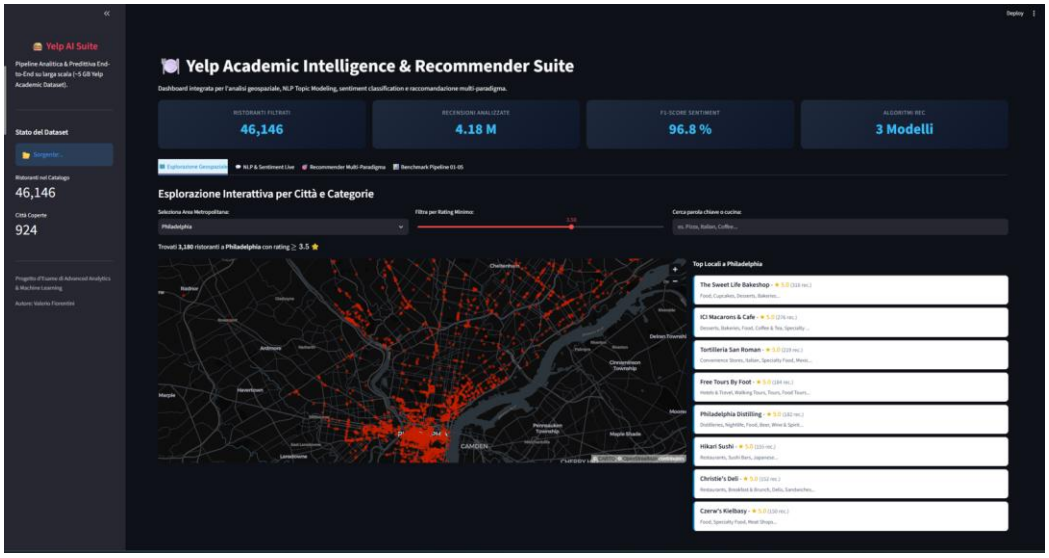


Figura 2 - *Dashboard Streamlit*: esplorazione geospaziale (Philadelphia)

4. Social Network Analysis

Le interazioni tra utenti sono state modellate come grafo con *NetworkX*. Da ciò si possono dedurre delle peculiarità:

- *Opinion leader*: gli utenti più influenti vengono estratti tramite *Degree Centrality*.
- Distribuzione *Power-Law*: l'80% degli utenti esegue al massimo 5 recensioni, mentre pochi utenti si dimostrano particolarmente attivi tanto da rappresentare gran parte dell'attività.
- *Community Detection*: il partizionamento con l'algoritmo di Louvain individua gruppi di utenti coesi.

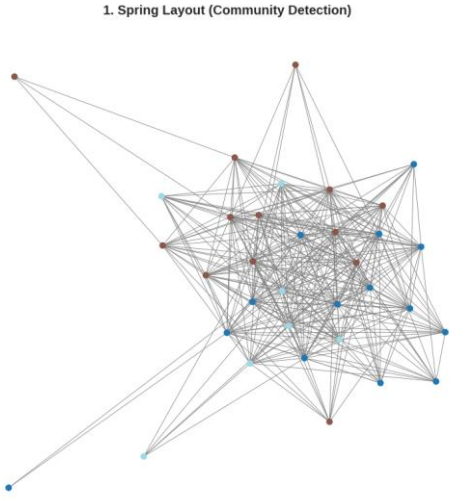


Figura 3 - Grafo della rete con *layout spring* (*community detection*)

5. Text mining e *topic modeling*

Mediante il *Natural Language Processing*, i testi delle recensioni vengono trasformati in r*app*resentazioni numeriche.

5.1 *TF-IDF*

La metrica *TF-IDF* converte il testo in matrici numeriche sparse e misura l'importanza di un termine t in un documento d all'interno di un corpus D:

$$TFIDF(t, d, D) = TF(t, d) \times log(|D| / (1 + DF(t, D)))$$

5.2 Classificazione del sentiment

Una *Logistic Regression*, addestrata sulla matrice *TF-IDF*, valuta il testo inserito dall'utente in modo positivo o negativo in tempo reale. Come modello di confronto, è stato scelto un *Multinomial Naive Bayes*.

Per quanto riguarda la gestione dello s*bi*lanciamento delle classi e la valutazione di equilibrio tra falsi positivi e falsi negativi, la validazione si fonda su *F1-score*:

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Il modello raggiunge un F1 di circa 0,97 con *AUC* pari a 0,985.

5.3 *Topic modeling* (LDA)

La *Latent Dirichlet Allocation* raggruppa le opinioni esposte dai clienti in cinque macro-temi: servizio, cibo, attese, prezzo e atmosfera. La combinazione di vari topic con i *rating* rende possi*bi*le comprendere come mai un determinato locale possa perdere stelle e dove sia necessario intervenire per implementare il servizio. Le *word cloud* evidenziano una stretta correlazione tra recensione positiva e atteggiamenti cortesi, nonché qualità del cibo, e, al contempo, una relazione altrettanto solida tra termini negativi e un servizio inefficiente e distaccato.



Figura 4 - *Word cloud* delle recensioni positive e negative

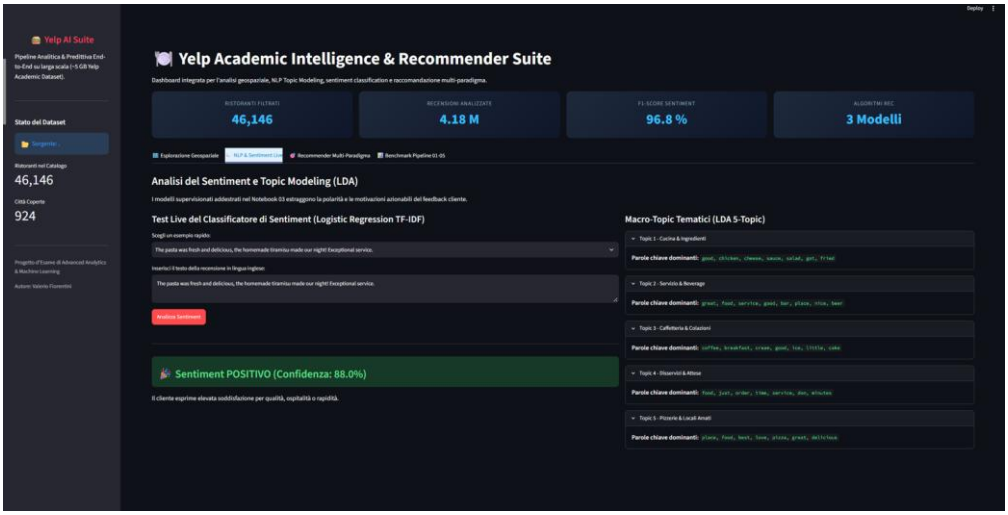


Figura 5 - *Dashboard Streamlit*: sentiment *live* e topic LDA

6. Motore di raccomandazione multi-paradigma

Il cuore del progetto si concretizza in un sistema ibrido fondato su tre approcci distinti, confrontati sia in termini di accuratezza sia di latenza.

6.1 *Content-Based Filtering*

Categorie e attributi del ristorante (tra i quali si possono annoverare il parcheggio, la cucina e l'atmosfera) vengono tradotti in uno spazio vettoriale. Inoltre, la vicinanza tra due locali A e B si misura con la *Cosine Similarity*, ossia il coseno dell'angolo tra i loro vettori:

$$sim(A, B) = cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

In tal caso, il numeratore r*app*resenta la sovr*app*osizione diretta degli attributi, mentre il denominatore normalizza i vettori mitigando l'effetto della loro lunghezza. Il valore è compreso tra 0 e 1. Il metodo è immune al problema del *cold-start* dato che non richiede lo storico dell'utente e garantisce latenze dell'ordine dei millisecondi.

6.2 *Collaborative Filtering* (*Truncated SVD*)

La matrice *User-Item* sparsa, che riporta gli utenti sulle righe e i ristoranti sulle colonne, viene scomposta in vettori singolari. Ciò riduce la dimensionalità e porta alla luce relazioni latenti, ovverosia *pattern* comportamentali condivisi tra utenti:

$$M \approx U \Sigma V^T$$

U contiene i fattori latenti degli utenti, V^T quelli dei locali e Σ l'importanza di ciascun fattore.

6.3 *Deep Learning* (*Neural Collaborative Filtering*)

Con *Keras* si addestrano strati di *embedding* che *app*rendono rappresentazioni dense e non lineari sia degli utenti sia dei locali. L'addestramento minimizza l'errore quadratico medio, valutato tramite *RMSE*:

$$RMSE = \sqrt{((1/n) \Sigma (y_i - \hat{y}_i)^2)}$$

Il modello stima con grande precisione le stelle che un utente assegnerebbe a un locale, al prezzo di una latenza maggiore.

Paradigma	Tecnica	Punto di forza	Latenza
Content-Based	Cosine Similarity (KNN)	Nessun cold-start	~55 ms
Collaborative Filtering	Truncated SVD	Relazioni latenti tra utenti	n.d.
Deep Learning	Keras Embeddings (NCF)	Massima accuratezza	~436 ms

Tabella 2 - Confronto dei tre paradigmi di raccomandazione

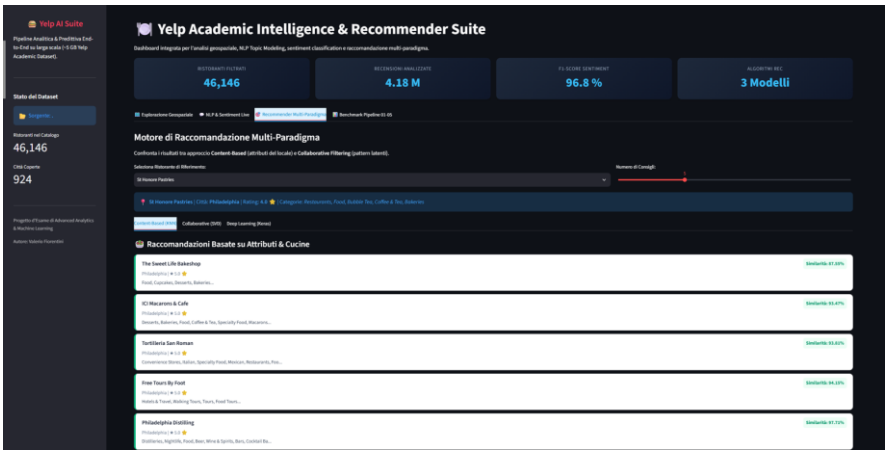


Figura 6 - *Dashboard Streamlit*: motore di raccomandazione multi-paradigma

7. Interfaccia utente (*dashboard* web)

Per rendere accessibile l'intero ecosistema è stata sviluppata una *dashboard* interattiva in *Streamlit*, con le seguenti caratteristiche:

- Dinamismo *live*: ogni interazione, come la selezione di un profilo dal menù a tendina, aggiorna il *DataFrame* e rigenera subito le raccomandazioni.
- Caching (@st.cache_data): modelli serializzati (*.joblib*), reti neurali e file *Parquet* vengono caricati in memoria solo al primo avvio; le richieste successive usano la cache con latenze quasi nulle.
- Visualizzazione multi-dominio: *tab* dedicati a mappe *GIS* esplorabili, *tester* del sentiment in tempo reale, comparatore dei *recommender* e plancia di *benchmark*.

8. Risultati e *benchmark*

Ambito	Metrica	Valore
Gestione dati	Compressione (JSON → Parquet)	>95%
Sentiment (Logistic Regression)	F1-score	0,97
Sentiment (Logistic Regression)	AUC	0,985
Recommender neurale	MAE	0,76 (errore medio < 1 star)
Recommender neurale	RMSE	0,94
Latenza KNN	Tempo di risposta	~55 ms
Latenza Deep Learning	Tempo di risposta	~436 ms

Tabella 3 - Sintesi delle metriche principali

Il *trade-off* tra latenza e accuratezza guida la scelta del modello: il *KNN*, con circa 55 ms di risposta, è ideale per *app*licazioni in tempo reale, mentre il *Deep Learning* impiega circa 436 ms sebbene offra previsioni decisamente più precise. L'integrazione dei due *app*rocci nella medesima *dashboard* consente di scegliere il compromesso più adatto al contesto d'uso.

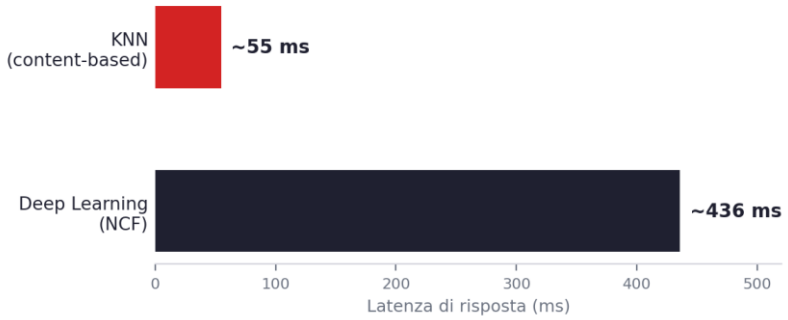


Figura 7 - Latenza di risposta: *KNN* contro *Deep Learning*

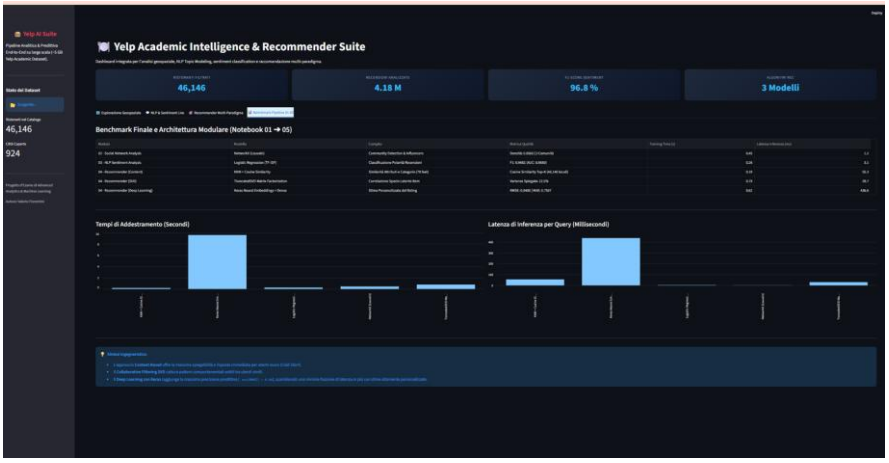


Figura 8 - *Dashboard Streamlit*: *benchmark* della *pipeline* (moduli 01-05)

9. Conclusioni

Il progetto ha permesso di *app*licare con successo le tecniche di Sistemi Intelligenti per Internet, dalla *Text Acquisition* all'*Information Retrieval* e alla *Social Search*. Dunque, l'integrazione di *NLP* e *Recommender Systems* all'interno di un'interfaccia interattiva garantisce un prodotto utile sia ai clienti in cerca di locali sia ai manager interessati alla *business intelligence*.

Yelp AI Suite unisce l'immediatezza della *Cosine Similarity* alla *precisione* delle reti neurali, offrendo un'esperienza d'uso reattiva e un quadro analitico completo, dall'*ingestion* alla *BI*.

Riferimenti

- Yelp Open *Dataset(JSON)*: https://business.yelp.com/data/resources/open-dataset/
- Repository del progetto: https://github.com/valeriofiorentini/yelp-bigdata-mining-recommender